

Jahangirnagar University

Department of Computer Science and Engineering

2nd Year 1st Semester B.Sc. (Hons.) Final

Examination – 2022

Course: CSE 209 – Algorithms-I*Complete Answer Script***Contents**

Section-1 (CO1)	2
Section-2 (CO2)	4
Section-3 (CO3)	7
Section-4 (CO4)	10
Section-5 (CO5)	14

SECTION-1 (CO1)

Seven questions; answer any six.

(a) Two Differences: Greedy vs. Dynamic Programming

Greedy Strategy	Dynamic Programming (DP)
Makes a locally optimal (greedy) choice at each step without reconsidering past decisions.	Solves all sub-problems and combines their solutions; may revisit sub-problems via stored results.
Does <i>not</i> guarantee a globally optimal solution in general; works only when the problem has the <i>greedy-choice property</i> .	Always yields a globally optimal solution when the problem has <i>optimal substructure</i> and <i>overlapping sub-problems</i> .

(b) Complexity of Sorting Algorithms

Algorithm	Best Case	Worst Case
Linear Search	$\Omega(1)$	$O(N)$
Insertion Sort	$\Omega(N)$ (already sorted)	$O(N^2)$ (reverse sorted)
Quick Sort	$\Omega(N \log N)$ (balanced pivot)	$O(N^2)$ (sorted, naive pivot)
Heap Sort	$\Omega(N \log N)$	$O(N \log N)$

(c) Properties of a Binary Search Tree (BST)

1. Every node has at most **two children** (left and right).
2. The **left subtree** of a node contains only nodes with keys *less than* the node's key.
3. The **right subtree** of a node contains only nodes with keys *greater than* the node's key.
4. Both the left and right subtrees are themselves valid BSTs (recursive definition).
5. An **in-order traversal** of a BST yields keys in sorted (non-decreasing) order.

(d) Divide and Conquer Paradigm

Divide and Conquer solves a problem by:

1. **Divide:** Split the problem into two or more smaller sub-problems of the same type.
2. **Conquer:** Solve each sub-problem recursively; solve base cases directly.
3. **Combine:** Merge/combine the sub-problem solutions to get the solution to the original problem.

Classic examples: Merge Sort, Quick Sort, Binary Search, Strassen's Matrix Multiplication.

(e) Properties of a Dynamic Programming Algorithm

1. **Optimal Substructure:** An optimal solution to the problem contains optimal solutions to sub-problems.
2. **Overlapping Sub-problems:** The same sub-problems recur many times; DP stores their results to avoid recomputation (memoisation or tabulation).
3. **Bottom-up or Top-down:** Solutions can be built iteratively from the smallest sub-problems (tabulation) or recursively with caching (memoisation).
4. **Polynomial Complexity:** By reusing stored results, DP typically reduces exponential brute-force to polynomial time.

(f) Proof: Dijkstra's Complexity is $O((V + E) \log_2 V)$

Dijkstra's algorithm using a **binary min-heap** priority queue:

- **Initialisation:** Insert all V vertices into the heap. Each **insert** costs $O(\log V)$. Total: $O(V \log V)$.
- **Extract-Min:** Performed V times (once per vertex). Each costs $O(\log V)$. Total: $O(V \log V)$.
- **Decrease-Key (Relax):** Performed at most once per edge (E times total). Each heap decrease-key costs $O(\log V)$. Total: $O(E \log V)$.

Overall:

$$T = O(V \log V) + O(V \log V) + O(E \log V) = O((V + E) \log_2 V)$$

For connected graphs $E \geq V - 1$, so the dominant term is $O(E \log V)$. *Proved.*

(g) Tree Edge, Forward Edge, Cross Edge, Back Edge (DFS)

These are the four types of edges classified during a Depth-First Search (DFS):

- **Tree Edge:** An edge (u, v) where v is first discovered by exploring u . These form the DFS forest.
- **Forward Edge:** An edge (u, v) where v is a *descendant* of u in the DFS tree but (u, v) is *not* a tree edge (i.e., v was already discovered).
- **Back Edge:** An edge (u, v) where v is an *ancestor* of u in the DFS tree. A back edge indicates a **cycle** in a directed graph.
- **Cross Edge:** An edge (u, v) where v is neither an ancestor nor a descendant of u — it goes between different branches or DFS trees.

Note: In an undirected graph, DFS produces only tree edges and back edges.

SECTION-2 (CO2)

Four questions; answer any three.

(a) Merge Sort on $\{8, 5, 7, 1, 6, 4, 3\}$

Step-by-step (2-way Merge Sort):

Level 0 (original): $[8, 5, 7, 1, 6, 4, 3]$

Split:

Left: $[8, 5, 7, 1]$ Right: $[6, 4, 3]$

Split Left:

$[8, 5]$ and $[7, 1]$

$[8][5]$ $[7][1]$

Merge $[8][5] \rightarrow [5, 8]$

Merge $[7][1] \rightarrow [1, 7]$

Merge $[5, 8] + [1, 7] \rightarrow [1, 5, 7, 8]$

Split Right:

$[6]$ and $[4, 3]$

$[4][3]$

Merge $[4][3] \rightarrow [3, 4]$

Merge $[6] + [3, 4] \rightarrow [3, 4, 6]$

Final Merge: $[1, 5, 7, 8] + [3, 4, 6] \rightarrow [1, 3, 4, 5, 6, 7, 8]$

Sorted result: $[1, 3, 4, 5, 6, 7, 8]$

Complexity via Recursion Tree: The recurrence is $T(n) = 2T(n/2) + O(n)$. By the Master Theorem (case 2): $T(n) = \Theta(n \log n)$.

The recursion tree has $\log_2 n$ levels; at each level the total work is $O(n)$ (merge step). Hence overall: $\boxed{O(n \log n)}$.

(b) Quick Sort on $\{3, 5, 2, 4, 7, 6, 1\}$ — Leftmost Element as Pivot

Iteration 1: Pivot = 3 (leftmost). Partition around 3:

Array: $[3, 5, 2, 4, 7, 6, 1]$

Elements < 3 : $\{2, 1\}$, Elements > 3 : $\{5, 4, 7, 6\}$

After partition: $\underbrace{[2, 1]}_{\text{left}}, 3, \underbrace{[5, 4, 7, 6]}_{\text{right}}$

(Only first iteration shown as required by the question.)

Continuing recursively:

- Left $[2, 1]$: pivot = 2, gives $[1, 2]$
- Right $[5, 4, 7, 6]$: pivot = 5, gives $[4, 5, 7, 6] \rightarrow$ right $[7, 6]$: pivot = 7, gives $[6, 7]$

Final sorted: [1, 2, 3, 4, 5, 6, 7]

Randomized Quick Sort vs. Quick Sort:

Choose Randomized Quick Sort.

- Standard Quick Sort with a fixed pivot (e.g., leftmost) degrades to $O(n^2)$ on already-sorted or reverse-sorted inputs.
- Randomized Quick Sort selects the pivot *uniformly at random*, making worst-case behaviour extremely unlikely. Its **expected** time complexity is $O(n \log n)$ regardless of input order.
- The probability of consistently bad pivots decreases exponentially with n , giving near-guaranteed $O(n \log n)$ performance.

(c) Algorithm Enigma Analysis

```

ALGORITHM Enigma(A[0..n-1, 0..n-1])
//Input: A matrix A[0..n-1, 0..n-1] of real numbers
for i <- 0 to n-2 do
  for j <- i+1 to n-1 do
    If A[i, j] != A[j, i]
      return false
return true

```

(i) **Basic Operation:** The **comparison** $A[i, j] \neq A[j, i]$ is the basic (elementary) operation, executed inside the innermost loop.

(ii) **Number of times the basic operation is executed:**

The outer loop runs $i = 0, 1, \dots, n-2$ (i.e., $n-1$ times).

For each i , the inner loop runs $j = i+1, i+2, \dots, n-1$, i.e., $n-1-i$ times.

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$$

$$C(n) = \frac{n(n-1)}{2} = \Theta(n^2)$$

What does the algorithm do? It checks whether the matrix A is **symmetric** (i.e., $A[i, j] = A[j, i]$ for all $i \neq j$). Returns **true** if symmetric, **false** otherwise.

(d) Single-Source Shortest Path from Node a

Graph (from figure): Vertices $\{a, b, c, d, e, f, g\}$ with weighted edges including some negative weights.

Since the graph contains **negative edge weights**, **Dijkstra's algorithm is not appropriate**. The correct algorithm is **Bellman-Ford**.

Bellman-Ford Algorithm:

1. Initialise $d[a] = 0$, all other distances $= \infty$.
2. Relax all edges $V-1$ times.

3. Check for negative-weight cycles (one more relaxation pass).

Reading the graph (approximate weights from figure): Edges: $a \rightarrow b$ (9), $a \rightarrow e$ (8), $b \rightarrow c$ (10), $b \rightarrow d$ (4), $d \rightarrow c$ (7), $d \rightarrow g$ (5), $d \rightarrow f$ (2), $e \rightarrow d$ (2), $g \rightarrow c$ (6), $f \rightarrow g$ (1), $e \rightarrow f$ (-), etc.

Iteration 1:

$$d[b] = \min(\infty, 0 + 9) = 9$$

$$d[e] = \min(\infty, 0 + 8) = 8$$

$$d[d] = \min(\infty, 8 + 2) = 10$$

$$d[f] = \min(\infty, 10 + 2) = 12$$

$$d[g] = \min(\infty, 10 + 5) = 15, \quad \min(15, 12 + 1) = 13$$

$$d[c] = \min(\infty, 9 + 10) = 19, \quad \min(19, 10 + 7) = 17$$

After $V - 1$ iterations, the shortest distances from a converge. The algorithm correctly handles negative weights and detects negative cycles.

Complexity of Bellman-Ford: $O(VE)$

SECTION-3 (CO3)

Four questions; answer any three.

(a) Interval Scheduling — Minimize Number of Classrooms

Problem: Given a set of lectures with start and finish times, schedule all lectures minimising the number of classrooms such that no two lectures in the same room overlap.

Optimal Strategy (Greedy — Interval Graph Colouring):

1. Sort all lectures by **start time**.
2. Maintain a min-heap of classrooms ordered by their **earliest finish time**.
3. For each lecture ℓ (in start-time order):
 - If the classroom with the earliest finish time $f \leq s(\ell)$ (start of ℓ), assign ℓ to that room and update its finish time.
 - Otherwise, open a **new classroom** and assign ℓ to it.
4. The total number of classrooms used equals the maximum number of lectures overlapping at any instant (the **depth** of the schedule).

From the figure (reading approximate intervals):

Lectures (start, finish): (0, 6), (1, 4), (3, 5), (4, 7), (5, 9), (6, 10), (7, 11) (approximate from graph).

Sorted by start: same order.

- Room 1: Lecture starts at 0 (finish 6).
- Lecture at 1: overlaps Room 1 $\rightarrow$ Room 2 (finish 4).
- Lecture at 3: overlaps both $\rightarrow$ Room 3 (finish 5).
- Lecture at 4: Room 2 finishes at $4 \leq 4 \rightarrow$ reuse Room 2 (finish 7).
- Lecture at 5: Room 3 finishes $5 \leq 5 \rightarrow$ reuse Room 3 (finish 9).
- Lecture at 6: Room 1 finishes $6 \leq 6 \rightarrow$ reuse Room 1 (finish 10).
- Lecture at 7: Room 2 finishes $7 \leq 7 \rightarrow$ reuse Room 2 (finish 11).

Minimum classrooms required = 3.

Key Insight: The minimum number of classrooms equals the maximum depth (maximum simultaneous overlaps) of the interval schedule.

(b) Binary Search Tree for $S = \{10, 22, 9, 33, 21, 50, 41, 60, 80\}$

Insert elements in the given order:

Insert 10: root = 10
 Insert 22: 22 > 10, go right
 Insert 9: 9 < 10, go left
 Insert 33: 33 > 10 → right (22) → 33 > 22, go right
 Insert 21: 21 > 10 → right (22) → 21 < 22, go left
 Insert 50: > 10 → > 22 → > 33, go right
 Insert 41: > 10 → > 22 → > 33 → 41 < 50, go left
 Insert 60: > 10 → > 22 → > 33 → > 50, go right
 Insert 80: > 10 → > 22 → > 33 → > 50 → > 60, go right

BST Structure:

Level 0: 10
 Level 1: 9 (left), 22 (right)
 Level 2: 21 (left of 22), 33 (right of 22)
 Level 3: 50 (right of 33)
 Level 4: 41 (left of 50), 60 (right of 50)
 Level 5: 80 (right of 60)

In-order traversal (verification): 9, 10, 21, 22, 33, 41, 50, 60, 80 — sorted. ✓

(c) LCS of $X = \text{“ABCBDAB”}$ and $Y = \text{“BDCABA”}$

Note: The question states $X = \text{“ABCBDAB”}$ and $Y = \text{“BDCABA”}$ (reading “BDCABA” from “BDCABA”).

DP Recurrence:

$$c[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ c[i-1][j-1] + 1 & X[i] = Y[j] \\ \max(c[i-1][j], c[i][j-1]) & X[i] \neq Y[j] \end{cases}$$

DP Table ($X = \text{ABCBDAB}$ rows, $Y = \text{BDCABA}$ columns):

	ϵ	B	D	C	A	B	A
ϵ	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

LCS Length = 4.

One LCS: “BCBA” or “BDAB” (length 4).

Back-tracking: Starting at $c[7][6] = 4$: $X[7] = B = Y[6] \rightarrow$ diagonal: $c[6][5] = 3$; $X[6] = A = Y[5] \rightarrow$ diagonal: $c[5][4]$... tracing gives **BCBA**.

Complexity: $O(mn)$ time and space, $m = |X| = 7$, $n = |Y| = 6$.

(d) Chinese Remainder Theorem (CRT)**System of congruences:**

$$x \equiv 6 \pmod{11}, \quad x \equiv 13 \pmod{16}, \quad x \equiv 9 \pmod{21}, \quad x \equiv 19 \pmod{25}$$

Check pairwise coprimality: $\gcd(11, 16) = 1$, $\gcd(11, 21) = 1$, $\gcd(11, 25) = 1$,
 $\gcd(16, 21) = 1$, $\gcd(16, 25) = 1$, $\gcd(21, 25) = 1$. ✓ All moduli are pairwise coprime.

Apply CRT:

$$N = 11 \times 16 \times 21 \times 25 = 92400$$

$$N_1 = \frac{92400}{11} = 8400, \quad N_2 = \frac{92400}{16} = 5775, \quad N_3 = \frac{92400}{21} = 4400, \quad N_4 = \frac{92400}{25} = 3696$$

Find modular inverses $M_i = N_i^{-1} \pmod{m_i}$ using Extended Euclidean Algorithm:

M_1 : $8400 \pmod{11} = 8400 - 763 \times 11 = 8400 - 8393 = 7$. Need $7^{-1} \pmod{11}$. $7 \times 8 = 56 \equiv 1 \pmod{11}$. So $M_1 = 8$.

M_2 : $5775 \pmod{16} = 5775 - 360 \times 16 = 5775 - 5760 = 15$. Need $15^{-1} \pmod{16}$.
 $15 \times 15 = 225 = 14 \times 16 + 1 \equiv 1 \pmod{16}$. So $M_2 = 15$.

M_3 : $4400 \pmod{21} = 4400 - 209 \times 21 = 4400 - 4389 = 11$. Need $11^{-1} \pmod{21}$.
 $11 \times 2 = 22 \equiv 1 \pmod{21}$. So $M_3 = 2$.

M_4 : $3696 \pmod{25} = 3696 - 147 \times 25 = 3696 - 3675 = 21$. Need $21^{-1} \pmod{25}$.
 $21 \times 6 = 126 = 5 \times 25 + 1 \equiv 1 \pmod{25}$. So $M_4 = 6$.

Solution:

$$\begin{aligned} x &= (r_1 N_1 M_1 + r_2 N_2 M_2 + r_3 N_3 M_3 + r_4 N_4 M_4) \pmod{N} \\ &= (6 \times 8400 \times 8 + 13 \times 5775 \times 15 + 9 \times 4400 \times 2 + 19 \times 3696 \times 6) \pmod{92400} \\ &= (403200 + 1126125 + 79200 + 421344) \pmod{92400} \\ &= 2029869 \pmod{92400} \\ &= 2029869 - 21 \times 92400 = 2029869 - 1940400 \\ &= \boxed{89469} \end{aligned}$$

Verify: $89469 \pmod{11} = 89469 - 8133 \times 11 = 89469 - 89463 = 6$ ✓
 $89469 \pmod{16} = 89469 - 5591 \times 16 = 89469 - 89456 = 13$ ✓
 $89469 \pmod{21} = 89469 - 4260 \times 21 = 89469 - 89460 = 9$ ✓
 $89469 \pmod{25} = 89469 - 3578 \times 25 = 89469 - 89450 = 19$ ✓

SECTION-4 (CO4)

Four questions; answer any three.

(a) Huffman Coding Application & Codeword Length

Application of Huffman Coding in Computer Science: Huffman coding is used for **lossless data compression**. It builds a variable-length prefix-free code where more frequent symbols receive shorter codes, minimising the total encoded length. Applications include: ZIP/GZIP file compression, JPEG/PNG image formats, MP3 audio, and HTTP/2 header compression.

Given frequencies (thousands):

Character	a	b	c	d	e	F
Frequency	34	14	12	16	14	10

Build Huffman Tree (Min-Heap):

Step 1: Sort by frequency: $F(10), c(12), b(14), e(14), d(16), a(34)$

Step 2: Merge two smallest: $F(10) + c(12) = Fc(22)$

Queue: $b(14), e(14), d(16), Fc(22), a(34)$

Step 3: Merge: $b(14) + e(14) = be(28)$

Queue: $d(16), Fc(22), be(28), a(34)$

Step 4: Merge: $d(16) + Fc(22) = dFc(38)$

Queue: $be(28), a(34), dFc(38)$

Step 5: Merge: $be(28) + a(34) = bea(62)$

Queue: $dFc(38), bea(62)$

Step 6: Merge: $dFc(38) + bea(62) = \text{root}(100)$

Codewords (left = 0, right = 1):

Character	Frequency	Code	Length
a	34	11	2
b	14	100	3
e	14	101	3
d	16	00	2
c	12	011	3
F	10	010	3

Average codeword length:

$$L = \frac{34 \times 2 + 14 \times 3 + 14 \times 3 + 16 \times 2 + 12 \times 3 + 10 \times 3}{100} = \frac{68 + 42 + 42 + 32 + 36 + 30}{100} = \frac{250}{100} = 2.5$$

(b) MST using Prim's and Kruskal's — Graph with vertices $\{a, b, c, d, e, f\}$

Graph edges (from figure): $a-b$: 5, $a-c$: 9, $a-e$: 1, $b-d$: 8, $b-f$: 3 (approx, reading from figure), $c-d$: 3, $d-f$: 7, $e-f$: 2, $c-f$ or other edges as shown.

Prim's Algorithm (starting from a):

1. Start: $\{a\}$, choose cheapest edge from a : $a-e$ (1) $\rightarrow$ MST = $\{a, e\}$
2. From $\{a, e\}$: cheapest outgoing = $e-f$ (2) $\rightarrow$ MST = $\{a, e, f\}$
3. From $\{a, e, f\}$: cheapest = $f-b$ or $f-c$, say $b-f$ (3) $\rightarrow$ MST = $\{a, e, f, b\}$
4. Cheapest remaining: $c-d$ (3) $\rightarrow$ MST = $\{a, e, f, b, c\}$... and so on until all vertices included.

Kruskal's Algorithm:

1. Sort all edges by weight: $(a, e, 1), (e, f, 2), (b, f, 3), (c, d, 3), (a, b, 5), (d, f, 7), (b, d, 8), (a, c, 9)$
2. Add (a, e) : no cycle. Add.
3. Add (e, f) : no cycle. Add.
4. Add (b, f) or (c, d) : no cycle. Add both.
5. Add (a, b) : check — would a, b form cycle? $a-e-f-b$ exists, so YES — skip.
6. Continue until $|V| - 1 = 5$ edges added.

Distinguishing Prim's vs Kruskal's:

Prim's Algorithm	Kruskal's Algorithm
Time Complexity: $O(E \log V)$ with binary heap; $O(E + V \log V)$ with Fibonacci heap.	Time Complexity: $O(E \log E)$ for sorting edges = $O(E \log V)$.
Preferred Graph: Dense graphs (many edges, $E \approx V^2$) — fewer operations on adjacency matrix/list. Grows the MST as a single connected component from a starting vertex.	Preferred Graph: Sparse graphs (few edges, $E \approx V$) — quick edge sort and union-find. Builds MST by adding globally cheapest edges, possibly connecting disjoint components.

(c) 0/1 Knapsack, Capacity $W = 8$

Items:

Item	Weight	Price (Value)
A	2	7
B	1	8
C	5	14
D	2	5
E	4	10
F	3	15

DP Table $dp[i][w]$ = maximum value using items 1.. i with capacity w :
Rows = items A,B,C,D,E,F; Columns = capacity 0..8.

Item\w	0	1	2	3	4	5	6	7	8
0 (none)	0	0	0	0	0	0	0	0	0
A (2,7)	0	0	7	7	7	7	7	7	7
B (1,8)	0	8	8	15	15	15	15	15	15
C (5,14)	0	8	8	15	15	14	22	22	29
D (2,5)	0	8	8	15	15	15	22	22	29
E (4,10)	0	8	8	15	15	18	22	25	29
F (3,15)	0	8	8	15	23	23	30	30	37

Optimal value = 37

Back-tracking to find items:

$dp[F][8] = 37$: Include F (weight 3, price 15). Remaining capacity = 5, $dp[E][5] = 18$.
 $dp[E][5] = 18$: Include E (weight 4, price 10). Remaining capacity = 1, $dp[D][1] = 8$.
 $dp[D][1] = 8$: Skip D. $dp[C][1] = 8$: Skip C. $dp[B][1] = 8$: Include B (weight 1, price 8).
 Remaining = 0.

Selected items: F + E + B, weights = $3 + 4 + 1 = 8 \leq 8$ ✓, total price = $15 + 10 + 8 = 33$...

Re-checking row F, w=8: $\max(dp[E][8], dp[E][8-3] + 15) = \max(29, dp[E][5] + 15) = \max(29, 18 + 15) = \max(29, 33) = 33$.

Corrected table (row F): $dp[F][4] = \max(dp[E][4], dp[E][1] + 15) = \max(15, 8 + 15) = 23$

$dp[F][5] = \max(18, dp[E][2] + 15) = \max(18, 8 + 15) = 23$

$dp[F][6] = \max(22, dp[E][3] + 15) = \max(22, 15 + 15) = 30$

$dp[F][7] = \max(25, dp[E][4] + 15) = \max(25, 15 + 15) = 30$

$dp[F][8] = \max(29, dp[E][5] + 15) = \max(29, 18 + 15) = \mathbf{33}$

Optimal value = 33, achieved by items **B + F + E** (weights $1 + 3 + 4 = 8$, prices $8 + 15 + 10 = 33$).

Arrows in DP table indicate whether an item was included ($\nearrow$ diagonal) or excluded ($\uparrow$ up).

Complexity: $O(nW)$ where $n = 6$ items, $W = 8$. This is pseudo-polynomial.

(d) Activity Selection — Maximum Non-Overlapping Activities

Given:

Activity	1	2	3	4	5	6	7	8
Start	1	0	1	4	2	5	3	4
Finish	3	4	2	6	9	8	5	5

Greedy Strategy: Sort by **finish time**, then greedily select compatible activities.

Sorted by finish time:

Activity	Start	Finish
3	1	2
1	1	3
2	0	4
7	3	5
8	4	5
4	4	6
6	5	8
5	2	9

Selection:

1. Select Activity 3 (finish = 2). Last finish = 2.
2. Activity 1: start = 1 < 2 — skip.
3. Activity 2: start = 0 < 2 — skip.
4. Activity 7: start = 3 $\geq$ 2 — **select** (finish = 5). Last finish = 5.
5. Activity 8: start = 4 < 5 — skip.
6. Activity 4: start = 4 < 5 — skip.
7. Activity 6: start = 5 $\geq$ 5 — **select** (finish = 8). Last finish = 8.
8. Activity 5: start = 2 < 8 — skip.

Maximum non-overlapping activities: {3, 7, 6} — **total 3.**

Complexity: $O(n \log n)$ for sort + $O(n)$ for selection = $O(n \log n)$.

SECTION-5 (CO5)

Three questions; answer any two.

(a) Strongly Connected Components (SCC) using Kosaraju's Algorithm

Key Steps for Finding SCC:

1. Perform **DFS** on the original graph G ; record **finish times** for each vertex.
2. Compute the **transpose graph** G^T (reverse all edge directions).
3. Perform **DFS on G^T in decreasing order of finish times** from step 1.
4. Each DFS tree in step 3 is one **SCC**.

Graph (from figure): Vertices $\{a, b, c, d, e, f, g, h\}$

Edges: $a \rightarrow b, b \rightarrow c, b \rightarrow f, c \rightarrow d, c \rightarrow g, d \rightarrow h, d \rightarrow c$ (back), $e \rightarrow a, e \rightarrow f, f \rightarrow b$ (back?), $f \rightarrow g, g \rightarrow f$ (back?), $h \rightarrow d$ (back), $h \rightarrow g$ (approximate from figure).

Step 1: DFS on G from a :

DFS order: $a \rightarrow b \rightarrow c \rightarrow d \rightarrow h \rightarrow g \rightarrow f \rightarrow e$

Finish times (example): $f[g] = 1, f[f] = 2, f[h] = 3, f[d] = 4, f[c] = 5, f[b] = 6, f[e] = 7, f[a] = 8$

Step 2: Transpose G^T : Reverse all edges.

Step 3: DFS on G^T in order $a(8), e(7), b(6), c(5), d(4), h(3), f(2), g(1)$:

Starting from a : in G^T , a 's neighbours are those with edge to a in G , i.e., e . So reach $\{a, e\}$ — **SCC 1**.

From b (next unvisited): in G^T , reach $\{b, f, g\}$ — **SCC 2**.

From c : reach $\{c, d, h\}$ — **SCC 3**.

SCCs: $\{a, e\}, \{b, f, g\}, \{c, d, h\}$

Complexity of Kosaraju's Algorithm: Two DFS traversals on G and G^T , each $O(V + E)$, plus transpose $O(V + E)$.

Total: $O(V + E)$

(b) Negative Weight Cycle & Bellman-Ford

Statement: "If you have a cycle in the graph that has in sum a negative weight, then there is no shortest path."

Justification: TRUE. If a negative-weight cycle exists and is reachable from the source to a vertex v , then one can keep traversing the cycle, decreasing the path cost indefinitely. Thus, the shortest path distance to v would be $-\infty$, and no finite shortest path exists.

Graph (from figure): Vertices $\{s, t, x, y, z\}$

Edges: $s \rightarrow t$ (6), $s \rightarrow y$ (7), $t \rightarrow x$ (5), $t \rightarrow y$ (8), $t \rightarrow z$ (-4), $x \rightarrow t$ (-2), $y \rightarrow x$ (-3), $y \rightarrow z$ (9), $z \rightarrow x$ (7), $z \rightarrow s$ (2)

Bellman-Ford from s :

Initialise: $d[s] = 0$, all others $= \infty$.

Iteration 1:

$$\begin{aligned}
d[t] &= \min(\infty, 0 + 6) = 6 \\
d[y] &= \min(\infty, 0 + 7) = 7 \\
d[x] &= \min(\infty, 6 + 5) = 11, \quad \min(11, 7 + (-3)) = 4 \\
d[z] &= \min(\infty, 6 + (-4)) = 2 \\
d[x] &= \min(4, 2 + 7) = 4 \text{ (no improvement from } z) \\
d[t] &= \min(6, 4 + (-2)) = 2 \\
d[s] &= \min(0, 2 + 2) = 0 \text{ (no change)}
\end{aligned}$$

Iteration 2:

$$\begin{aligned}
d[t] &= \min(2, 0 + 6) = 2 \\
d[y] &= \min(7, 0 + 7) = 7 \\
d[x] &= \min(4, 2 + 5) = 4, \quad \min(4, 7 - 3) = 4 \\
d[z] &= \min(2, 2 - 4) = -2 \\
d[x] &= \min(4, -2 + 7) = 4 \\
d[t] &= \min(2, 4 - 2) = 2
\end{aligned}$$

Iteration 3:

$$\begin{aligned}
d[z] &= \min(-2, 2 - 4) = -2 \\
d[x] &= \min(4, -2 + 7) = 4 \text{ (check again)} \\
d[t] &= \min(2, 4 - 2) = 2
\end{aligned}$$

Iteration 4 ($V - 1 = 4$): No change.

Negative Cycle Check (5th iteration): Perform one more relaxation pass. If any distance decreases, a negative cycle exists.

Checking $z \rightarrow s \rightarrow t \rightarrow z$: $d[z] = d[s] + 6 + (-4) = 0 + 6 - 4 = 2 > -2$; $d[t] = d[x] - 2 = 4 - 2 = 2$ no change. **No negative cycle detected.**

Final shortest distances from s : $d[s] = 0$, $d[t] = 2$, $d[x] = 4$, $d[y] = 7$, $d[z] = -2$

Complexity of Bellman-Ford: $O(VE)$

(c) Radix Sort — Iterative Version

Setup: Array of n non-negative integers with values in range $[0, 2^{n^2})$.

(i) Time Complexity in terms of digit comparisons:

Each integer has at most $b = n^2$ bits (since max value $= 2^{n^2}$).

In Radix Sort, we process d digits (or passes), where with base (radix) r , $d = \lceil b / \log_2 r \rceil$.

Using **base- n** (radix $r = n$):

$$\text{Each digit is } \log_2 n \text{ bits, } d = \frac{n^2}{\log_2 n} \text{ passes}$$

Each pass is a stable sort (e.g., Counting Sort) on n keys over n digit values: $O(n + n) = O(n)$.

Total time:

$$T = d \times O(n) = \frac{n^2}{\log_2 n} \times O(n) = O\left(\frac{n^3}{\log n}\right)$$

In terms of **number of digit comparisons**: each pass examines one digit per element (n comparisons per pass), over $d = n^2 / \log_2 n$ passes:

$$\text{Digit comparisons} = n \times \frac{n^2}{\log_2 n} = \frac{n^3}{\log_2 n}$$

$$\boxed{T = O\left(\frac{n^3}{\log n}\right)}$$

(ii) Radix Sort vs. Selection Sort:

Use Radix Sort.

- **Selection Sort** has time complexity $O(n^2)$ comparisons regardless of input. For n integers with values up to 2^{n^2} , Selection Sort makes $\Theta(n^2)$ element comparisons — each comparison involves integers of up to n^2 bits, so actual bit-operation cost could be $O(n^2 \cdot n^2) = O(n^4)$.
- **Radix Sort** with the analysis above costs $O(n^3 / \log n)$ digit comparisons, which is *asymptotically faster* than Selection Sort's $O(n^2)$ element comparisons (since each element comparison in Selection Sort on n^2 -bit numbers costs $O(n^2)$ bit operations, giving $O(n^4)$ total).
- Therefore, **Radix Sort** is significantly more efficient for this input range.